UiA

Group name and candidates: Group 9

Rikke Krauss

Ida Elise Fyhrie Jamissen

Jimmy Trinh

Ask Lootz

Madalitso Phiri Skjelnes

Course code	IS-309
Course name	Videregående databasesystemer
Course responsible	Rania Elgazzar and Pouria Akbarighatar
Deadline	30. April 23:59
Number of words (whole document)	6610
Project/product title	Final portfolio

Table of Contents

Figure list

Table list

Introduction

This project is a compilation of the three obligatory assignments after the group received conclusive feedback where we refined them. This document consists of three segments:

Assignment 1: Data Model and Design Choices
Assignment 2: Stored Procedures and Triggers
Assignment 3: Storage Structure, Data Dictionary, and Performance Analysis

Furthermore, these segments provide the database implementation and design choices, how we created the stored procedures in addition to triggers, and analyzed the storage structure, data dictionary and utilized a performance analysis. The group also discussed the necessities of what the final product of the Bcycle database system should look like, with justifications of the group's own reflection. These segments are therefore refined into a final portfolio.

A .tar file of the database including an SQL script of the whole process is in the ZIP file. Link to our repo is here: https://github.com/asklootz/IS-309-Bcycle-Assignment

Assignment 1: Data Model and Design Choices

1.1 Reflection on design choices

In designing the data model for the Bcycle-style bike-share system, our main priority was to separate stable, identifying information from time-dependent, operational status. This guided us to create distinct entities for **program**, **station**, **station_status**, **bike**, and **bike_status**, and then further refine the design around docks. Programs and stations are relatively static, while both station-level and bike-level status change continuously over time. By introducing dedicated status tables with timestamps, we ensure that historical data can be queried and audited, while keeping the core entities simple and focused on identity and relatively stable attributes.

A key design choice was to model individual docks in their own table (**station_dock** or equivalent) rather than only tracking availability at the station level. This allows us to store state per dock (whether a dock is occupied, accepting returns, or out of service) and link each dock to both a station and, optionally, a single bike. The practical benefit is that we can detect and react to problems at a very fine-grained level: if one dock is broken or offline, it does not mean the entire station has to be marked as unavailable. Instead, the station can remain active while one or more specific docks are flagged as down. This improves resilience, since the system can continue operating even when individual components fail, and it gives both operators and users a more accurate view of capacity and problems at a station.

Modeling docks as a separate entity also supports richer queries and operational decisions. We can count total docks, distinguish between usable and faulty docks, and analyze patterns such as which docks fail more often or are more heavily used. This may add some complexity but improves normalization.

1.2 Datatypes and Naming Conventions

1.2.1 Identifiers

The following identifiers are used across the data model: **program_id**, **station_id**, **dock_id**, **station_status_id**, **bike_type_id**, **bike_id**, **bike_status_id**, **user_id**, **trip_id**, **purchase_id**, and **membership_type**.

The formatting conventions applied to these identifiers are summarized in the table below:

Table 1. Formatting Conventions Table

Table	Variable	Formatting	Example	Comment
station	station_id	*program_name*\ + "_station_" + *incrementing int*	bergen_station_1	
station_dock	station_dock_id	*station_id* + “_” + *incrementing int*	bergen_station_1_1	
station_status	station_status_id	“station_status_” + *incrementing int*	station_status_1	
bike_type	bike_type_id	*bike_type* + “_” + *incrementing int*	electric_1	
bike	bike_id	*program_name* + “_bike_” + *incrementing int*	bergen_bike_1	
bike_status	bike_status_id	“bike_status_” + *incrementing int*	bike_status_1	

trip	trip_id	“trip_” + *incrementing int*	trip_1	
user	user_id	“user_” + *incrementing int*	user_1	
bought_membership	purchase_id	“purchase_” + *incrementing int*	purchase_1	

All identifiers serve as either primary keys (**PK**), foreign keys (**FK**), or both, depending on their role within a given table. Since identifiers must be unique and cannot be **NULL**, each one is generated as a distinct value within a defined range. This ensures that every record is unambiguously distinguishable from all others across the system.

Additionally, all identifiers are defined as **varchar**, consistent with the ER diagram. This accommodates structured naming conventions such as **bergen_bike_1**, where the identifier encodes meaningful information about the program or station it belongs to.

1.2.2 Descriptive Attributes

The following attributes are classified as descriptive: **name**, **surname**, **address**, **location**, **maker**, **model**, **bike_type**, **status**, **ermail**, **url**, **timezone**, **phone**, **street**, **city**, and **state**.

All descriptive attributes use the **varchar** data type, which provides flexibility without imposing artificial limits on text length. A predefined maximum length is still required for each field to ensure consistency and storage efficiency. These fields are not used for calculations, indexing logic, or constraint enforcement, and therefore do not require more restrictive data types.

1.2.3 Geographic Attributes

latitude and **longitude** represent the physical location of stations and bikes within the system. Both attributes use the **numeric** data type, which provides greater precision compared to floating-point alternatives. This is particularly important for

geographic coordinates, where small differences in value can represent significant real-world distances.

1.2.4 Availability, Battery, Count, and Capacity Attributes

The following attributes fall under this category: **capacity**, **available_docks**, **regular_bikes_available**, **electric_bikes_available, cargo_bikes_available**, **smart_bikes_available**, **battery, battery_start, battery_end, total_trips**, and **total_distance**.

These attributes use **integer** as their primary data type, as they represent discrete quantities where fractional values are not meaningful. Battery level, for instance, is expressed as a percentage and is therefore best represented as a whole number. Capacity and availability count similarly deal only with whole unit values.

1.2.5 Boolean Attributes

The attributes **is_occupied**, **is_accepting_returns**, **is_Accepting_Renting**, and **is_active** are all represented using the **boolean** data type, storing a value of either **true** or **false**. These flags indicate the current operational state of a dock, station, or membership at any given point in time.

1.2.6 Timestamp Attributes

The following attributes are time-based: **last_updated**, **creation_date**, **purchase_time**, **activation_time**, **expiration_time**, **start_time**, and **end_time**.

All time attributes use the **timestamp with time zone** data type. Time is central to this system, as status tables are explicitly time-dependent and require precise ordering and history tracking. Including time zone support ensures compatibility with multi-program operations that may span different geographic regions and time zones.

1.2.7 Measurement Attributes

trip_distance, **remaining_range**, and **trip_cost** use **numeric** as their data type. These attributes represent measured or calculated values produced during a trip or by bike in operation.

1.2.8 Duration Attribute

trip_duration represents the total length of a trip and uses the **interval** data type, since the duration between the start and end of a trip rather than a specific point in time. For instance, a trip lasting 45 minutes would be stored as 00:45:00. Using interval provides for arithmetic operations directly on the duration, such as calculating average trip duration across all users or summing total time spent riding.

1.3 Choices of Constraints

These are the constraints that are being utilized in the Bcycle database implementation.

1.3.1 NOT NULL

Most attributes across all tables are defined as **NOT NULL**, ensuring that critical data is always present when a record is inserted. This constraint simply defines that a column must not be the NULL value (PostgreSQL, 2026). This enforces data integrity at the database level and prevents incomplete records from being stored.

1.3.2 Nullable Exceptions

Certain attributes are intentionally left nullable based on business logic and system design decisions. These exceptions are as follows:

dock_id in bike — a bike may exist in the system without being physically assigned to a dock, for example when it is in transit or undergoing maintenance.

ride_cost in trip — trip cost is considered optional and may not always be applicable, depending on the membership type or promotional conditions.

1.3.3 CHECK

CHECK constraints allow you to specify that the value in a certain column must satisfy a Boolean value expression (PostgreSQL, 2026).

email — validated against a standard email format pattern to ensure correctly structured entries.

country_code — restricted to a two-character uppercase format, consistent with ISO 3166-1 alpha-2 country codes.

Additional **CHECK** constraints, such as validating that a user's date of birth represents a realistic age, may be considered and implemented at a later stage as requirements become clearer.

1.3.4 UNIQUE

UNIQUE constraints are applied to all primary keys, as every identifier must be distinct across the table. This ensures that the data contained in a column is unique among all the rows in the table (PostgreSQL, 2026). Beyond primary keys, **UNIQUE** constraints are also applied wherever a field must contain no duplicate values, such as email addresses in user records.

1.4 SQL Design Choices

Several SQL design choices have been considered in advance to ensure that the database can be efficiently and consistently implemented in PostgreSQL. These choices focus on how entities are translated into relational tables and how relationships and constraints are handled at the database level.

1.4.1 Mapping the ER Diagram to Relational Tables

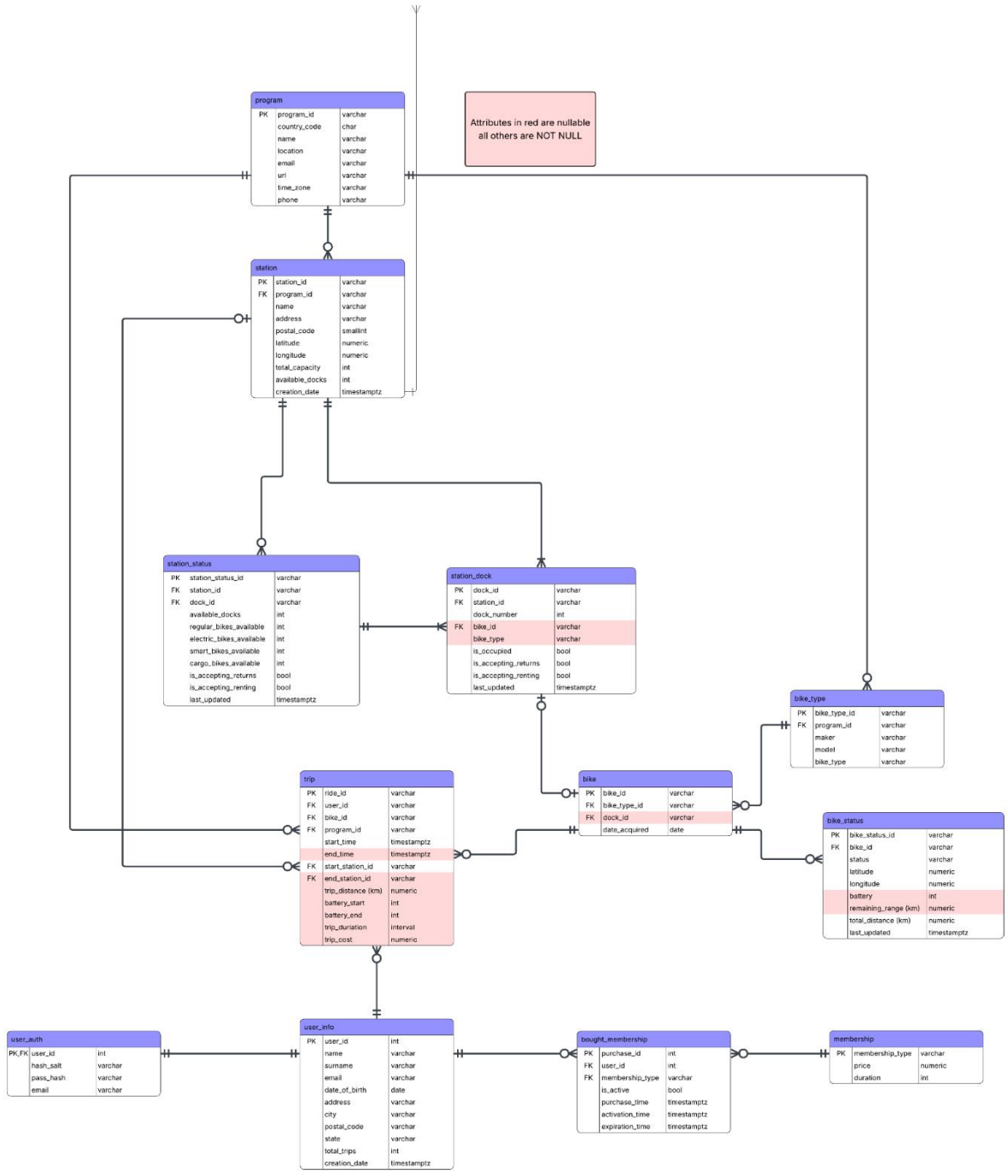

Figure 1. ER diagram for Bcycle database system

The entities that are defined in the ER diagram, such as **program**, **station**, **bike**, **user**, **trip**, **membership**, and related **status** entities, are intended to be implemented as a separate relational table. This approach follows design principles and ensures that real-world concepts are normalized and represented.

The primary keys defined in the ER diagram will be implemented using SQL primary key constraints. These keys identify records and provide a stable reference point for relationships between tables. The ER diagram specifies composite keys in some of the classes where uniqueness depends on a combination of attributes. The composite keys will be enforced directly in SQL to preserve the semantics of the design.

1.4.2 Identifier and Key Design Choices

The ER diagram uses meaningful identifiers instead of numeric keys. For example, **station** and **dock_id** follow a structured format that includes **program** and **station** information. This reflects how identifiers are used in real bike sharing systems. Using these identifiers as primary keys makes it easier to integrate external data and reduces the need for extra reference tables.

Moreover, **bike_type_id**, **bike_status_id**, **bike_id** and **station_status_id** lists **regular_1**, **electric_1**, **smart_1**, **cargo_1** (**bike_type_id**, depending which bike type), **bike_status_1** (**bike_status_id**), **bike_1** (**bike_iD**) and **station_status_1** (**station_status_id**). These identifiers provide their own unique naming conventions that are recognizable and in a simplistic way. Instead of using regular numbers, starting from 1, the group figured that it is better to have initial numbers with both integers and strings that represent its columns. That way we can distinguish their ID in clarity.

1.4.3 Choices of NULLable variables in the ER diagram

As shown in the ER-diagram most of the variables on the difference are set to **NOT NULL**. Most of the variables that are **NULL** are the ones that are the ones that will be set at a later time (**end_station**, **end_time** in **bergen.trip**), the ones that does not always have data as it's based on a certain condition (**dock_id** in **bergen.bike** and **bike_id** in **bergen.station_dock**).

These are not always true as they need to be physically connected for it to be data there and the final one being empty as we needed to have it in the assignment but no real use for it (**trip_cost** in **bergen**.**trip**). All the NULLable variables besides "**trip_cost**", are variable that will be set later or will not always- or cannot have data

due to the object's limitation (i.e. regular bike won't have battery percentage due to no battery).

1.4.4 Use of PL/pgSQL for Business Logic

While many integrity rules can be expressed using SQL constraints, some aspects of the system require procedural logic. This is particularly evident in the **bike_status** and **station_status** entities. These tables represent dynamic state derived from multiple factors, such as dock occupancy, bike availability, trip activity, and time-based updates.

PL/pgSQL triggers or functions are therefore planned to automatically maintain these status tables. For example, when a trip starts or ends, a trigger could update the corresponding bike's status, battery level, and location, as well as adjust the availability counts for the affected stations. Implementing this logic in PL/pgSQL ensures that these updates occur consistently and automatically, without relying on application-side code.

We plan to use database transactions when performing operations that affect multiple tables. For example, when a trip starts, updates must occur - a new trip must be created, the status must change, and the dock availability must be updated. If one update fails, the operation should be rolled back to prevent inconsistent data.

1.4.5 Rationale for Database-Level Logic

Placing business logic inside the database using PL/pgSQL provides a single source of truth for system rules. This approach reduces duplication of logic across multiple applications and ensures that all data modifications follow the same validation and update procedures. It also improves robustness, as rules are enforced even if data is modified directly at the database level.

Overall, the planned SQL and PL/pgSQL choices reflect the structure and semantics of the ER diagram while ensuring data integrity, consistency, and scalability. SQL is used to define tables, keys, relationships, and constraints, while PL/pgSQL is reserved for complex, dynamic business logic that cannot be easily expressed using declarative SQL alone. These design decisions provide a solid foundation for the future PostgreSQL implementation of the bike-sharing system.

Assignment 2: Stored Procedures and Triggers

2.1 CREATE_ACCOUNT_SP

The stored procedure CREATE_ACCOUNT_SP manages user creation in the Bcycle system, inserting data into both **bergen.user_info** and **bergen.user_auth** in a single operation. This returns the newly generated **user_id**.

2.1.1 Creating the stored procedure

Query Query History

```
CREATE OR REPLACE PROCEDURE bergen.create_account_sp(
    IN p_name character varying,
    IN p_surname character varying,
    IN p_email character varying,
    IN p_date_of_birth date,
    IN p_street character varying,
    IN p_city character varying,
    IN p_postal_code smallint,
    IN p_state character varying,
    IN p_hashsalt character varying,
    IN p_passhash character varying,
    IN p_user_id character varying)
LANGUAGE 'plpgsql'
    SECURITY DEFINER
    SET search_path=postgres, pg_temp
AS $BODY$
DECLARE
    v_user_id INT;
BEGIN
    INSERT INTO bergen.user_info (
        name, surname, email, date_of_birth,
        street, city, postal_code, state,
        total_trips, creation_date
    )
    VALUES (
        p_name, p_surname, p_email, p_date_of_birth,
        p_street, p_city, p_postal_code, p_state,
        0, CURRENT_TIMESTAMP
    )
    RETURNING user_id INTO v_user_id;

    INSERT INTO bergen.user_auth (
        user_id, hashsalt, passhash, email
    )
    VALUES (
        v_user_id, p_hashsalt, p_passhash, p_email
    );

    p_user_id := v_user_id;

    RAISE NOTICE 'Account created successfully with user_id: %', v_user_id;

EXCEPTION
    WHEN unique_violation THEN
        RAISE EXCEPTION 'An account with email % already exists.', p_email;
    WHEN OTHERS THEN
        RAISE EXCEPTION 'An error occurred: %', SQLERRM;
END;
$BODY$;
```

Data Output Messages Notifications

```
CREATE PROCEDURE

Query returned successfully in 97 msec.
```

Total rows: Query complete 00:00:00.097

Figure 2. Creating the stored procedure for CREATE_ACCOUNT_SP

2.1.2 Executing the stored procedure

Query | Query History

```
1  CALL bergen.create_account_sp(
2      'Jimmy', 'Test', 'jimmy.test@email.no',
3      '1998-12-08'::DATE,
4      'Bergengaten 1', 'Bergen', 5003::SMALLINT, 'Vestland',
5      'salt_jimmy123', 'hash_jimmy_001',
6      NULL
7  );
```

Data Output | Messages | Notifications

```
NOTICE:  Account created successfully with user_id: 18
CALL

Query returned successfully in 177 msec.
```

Figure 3. Executing the SP using CALL function

2.1.3 Verifying account creation

Figure 4. Query verification

2.2 INSERT_NEW_STATION

The stored procedure **insert_new_station** inserts a new station into the **bergen.station** table. It takes input parameters from the user when it is called to specify necessary data that cannot be set automatically. **station_id** is set automatically via its own called function that generates a new name id every time the procedure is called. It has feedback to tell the user if the procedure was successful or not.

2.2.1 Creating the stored procedure

```sql
-- Stored procedure that will create a new station filling in the remaining data
CREATE OR REPLACE PROCEDURE bergen.insert_new_station(  --(Ask)
    st_name VARCHAR,
    st_address VARCHAR,
    st_postal_code INT,
    st_latitude NUMERIC,
    st_longitude NUMERIC,
    st_capacity INT
)
LANGUAGE plpgsql
AS $$
DECLARE
    rows_affected INT;
BEGIN
    INSERT INTO bergen.station (station_id ,program_id, name, address, postal_code, latitude, longitude, total_capacity, available_docks)
    SELECT bergen.gen_station_id(name), program_id, st_name, st_address, st_postal_code, st_latitude, st_longitude, st_capacity, st_capacity
    FROM bergen.program
    WHERE program_id = 'bcycle_bergen';

    GET DIAGNOSTICS rows_affected = ROW_COUNT;

    IF rows_affected > 0 THEN
        RAISE NOTICE 'Suksess: % rad(er) satt inn.', rows_affected;
    ELSE
        RAISE WARNING 'Ingen rader satt inn! Sjekk om program_id "bcycle_bergen" eksisterer i bergen.program.';
    END IF;
END;
$$;
```

Data Output Messages Notifications

```
CREATE PROCEDURE

Query returned successfully in 103 msec.
```

Figure 5. Creating the SP for INSERT_NEW_STATION

2.2.2 Executing the stored procedure

Query Query History

```sql
CALL bergen.insert_new_station('Nygaten', 'Nygaten 1', 5003, 60.3920, 5.3210, 4);
```

Data Output Messages Notifications

```
NOTICE:  Suksess: 1 rad(er) satt inn.
CALL

Query returned successfully in 120 msec.
```

Figure 6. Calls the SP

2.2.3 Verifying station creation

```sql
select * from bergen.station;
```

Data Output Messages Notifications

Showing rows: 1 to 4 Page No: 1 of 1

	station_id [PK] character varying (100)	program_id character varying (100)	name character varying (30)	address character varying (50)	postal_code smallint	latitude numeric (9,7)	longitude numeric (9,7)	total_capacity integer	available_docks integer	creation_date timestamp with time zone (3)
1	bergen_station_1	bcycle_bergen	Bryggen	Bryggen 1	5003	60.3961000	5.3228000	4	2	2026-04-22 17:18:35.021+00
2	bergen_station_2	bcycle_bergen	Torgallmenningen	Torgallmenningen 1	5014	60.3925000	5.3240000	3	2	2026-04-22 17:18:35.021+00
3	bergen_station_3	bcycle_bergen	Byparken	Byparken 1	5015	60.3910000	5.3215000	3	2	2026-04-22 17:18:35.021+00
4	bergen_station_4	bcycle_bergen	Nygaten	Nygaten 1	5003	60.3920000	5.3210000	4	4	2026-04-27 08:46:26.988+00

Figure 7. Query verification

2.3 CREATE_BICYCLE_SP

This section provides an overview of the implementation of a stored procedure and related triggers for the bergen.bike table. The store procedure is responsible for inserting new bicycle records, including validation of input data and generation of a unique identifier. In addition, triggers are implemented to enforce data integrity.

2.3.1 Creating the stored procedure

```
-- Stored procedure that creates bikes
-- Stored procedure: create_bicycle_proc
CREATE OR REPLACE PROCEDURE bergen.create_bicycle_proc( --(Ida)
    IN p_bike_type_id VARCHAR
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_next_id INT;
    p_bike_id VARCHAR;
BEGIN
    -- Validate foreign key reference: bike type must exist
    PERFORM 1
    FROM bergen.bike_type
    WHERE bike_type_id = p_bike_type_id;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Invalid bike_type_id: % does not exist in bergen.bike_type',
    END IF;

    -- Generate bike_id according to required format
    SELECT COUNT(*) + 1
    INTO v_next_id
    FROM bergen.bike;

    p_bike_id := 'bergen_bike_' || v_next_id;
```

```
    -- Insert bicycle
    INSERT INTO bergen.bike (bike_id, dock_id, bike_type_id)
    VALUES (p_bike_id, NULL, p_bike_type_id);

EXCEPTION
    WHEN unique_violation THEN
        RAISE EXCEPTION 'Duplicate value error while creating bicycle.';
    WHEN raise_exception THEN
        RAISE;
    WHEN OTHERS THEN
        RAISE EXCEPTION 'Unexpected error in create_bicycle_proc: %', SQLERRM;
END;
$$;
```

Data Output | Messages | Notifications

```
CREATE PROCEDURE

Query returned successfully in 433 msec.
```

Figure 8. Creating the SP for CREATE_BICYCLE_SP

The screenshot shows that the stored procedure was successfully created in the database.

2.3.2 Trigger functions and triggers

```
86  -- This function will check that the date the bike was acquired is not in the future,
87  CREATE OR REPLACE FUNCTION bergen.check_date_acquired() --(Ida)
88  RETURNS TRIGGER
89  LANGUAGE plpgsql
90  AS $$
91  BEGIN
92      IF NEW.date_acquired > CURRENT_DATE THEN
93          RAISE EXCEPTION 'date_acquired cannot be in the future';
94      END IF;
95
96      RETURN NEW;
97  END;
98  $$;
99
100 -- Trigger object for trigger function
101 CREATE TRIGGER trg_check_date_acquired
102 BEFORE INSERT OR UPDATE ON bergen.bike
103 FOR EACH ROW
104 EXECUTE FUNCTION bergen.check_date_acquired();
105
```

Data Output | Messages | Notifications

```
CREATE TRIGGER

Query returned successfully in 108 msec.
```

```
105
106
107 -- Trigger function that will be used on direct insert-statements for adding to the "b
108 -- This function will simply raise a notice that an insert statement has been executed
109 CREATE OR REPLACE FUNCTION bergen.bike_insert_statement_trigger() --(Ida)
110 RETURNS TRIGGER
111 LANGUAGE plpgsql
112 AS $$
113 BEGIN
114     RAISE NOTICE 'Insert statement executed on bike table';
115     RETURN NULL;
116 END;
117 $$;
118
119 --Statement-level trigger
120 CREATE TRIGGER trg_bike_insert_statement
121 AFTER INSERT ON bergen.bike
122 FOR EACH STATEMENT
123 EXECUTE FUNCTION bergen.bike_insert_statement_trigger();
124
```

Data Output | Messages | Notifications

```
CREATE TRIGGER

Query returned successfully in 53 msec.
```

Figure 9. Creating a row-level trigger and a statement-level trigger

The screenshot shows that the trigger function and the trigger for both the row-level trigger and statement-level trigger were created successfully.

2.3.3 Verifying the insert

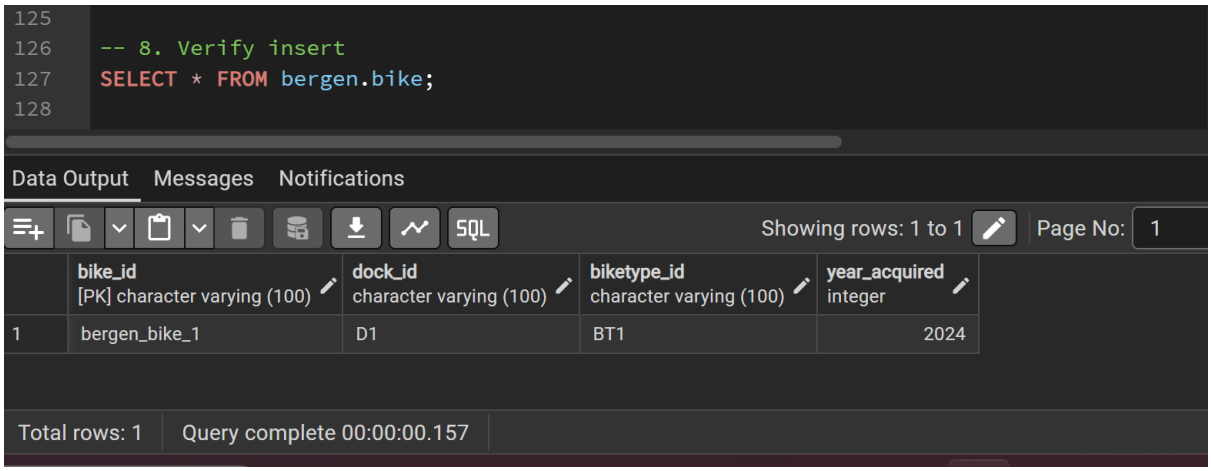

Figure 10. Query verification

The result for the **SELECT** query confirms that the bicycle was successfully inserted into the table.

2.4 PURCHASE MEMBERSHIP_SP

The stored procedure handles the registration of a new membership for an existing user in the system. It inserts a new record into the **bergen.bought_membership** table, connecting a user with a selected membership type and recording the purchase time. If a membership has not yet been activated, the expiration time remains **NULL** until activation occurs. The procedure returns the generated **purchase_id** as an **OUT** parameter, allowing the caller to reference the new membership record immediately.

2.4.1 Creating the stored procedure

```
53  -- Stored procedure that will create a purchase.
54  CREATE OR REPLACE PROCEDURE bergen.purchase_membership_proc( --(Rikke)
55      IN p_user_id VARCHAR,
56      IN p_membership_type VARCHAR,
57      IN p_is_active BOOLEAN,
58      IN p_activation_time TIMESTAMPTZ DEFAULT NULL,
59      INOUT p_purchase_id VARCHAR DEFAULT NULL
60  )
61  LANGUAGE plpgsql
62  AS $$
63  DECLARE
64      v_next_id INT;
65      v_duration INT;
66      v_activation_time TIMESTAMPTZ;
67      v_expiration_time TIMESTAMPTZ;
68  BEGIN
69      -- Validate that membership type exists
70      PERFORM 1
71      FROM bergen.membership
72      WHERE membership_type = p_membership_type;
73
74      IF NOT FOUND THEN
75          RAISE EXCEPTION 'Invalid membership type: % does not exist in bergen.membershi
```

```
75          RAISE EXCEPTION 'Invalid membership type: % does not exist in bergen.membershi
76      END IF;
77
78      -- Validate that user exists
79      PERFORM 1
80      FROM bergen.user_info
81      WHERE user_id = p_user_id;
82
83      IF NOT FOUND THEN
84          RAISE EXCEPTION 'User does not exist: user_id % was not found in bergen.user_i
85      END IF;
86
87          -- Generate purchase_id according to required format
88      SELECT COUNT(*) + 1
89      INTO v_next_id
90      FROM bergen.bought_membership;
91
92      p_purchase_id := 'purchase_' || v_next_id;
93
94      -- Get duration from membership table
95      SELECT duration
96      INTO v_duration
97      FROM bergen.membership
98      WHERE membership_type = p_membership_type;
```

```
99
100     -- Use current timestamp if activation time is not provided
101     v_activation_time := COALESCE(p_activation_time, CURRENT_TIMESTAMP);
102
103     -- Derive expiration time from activation_time + duration
104     v_expiration_time := v_activation_time + (v_duration || ' days')::INTERVAL;
105
106     -- Insert purchase
107     INSERT INTO bergen.bought_membership (
108         purchase_id,
109         user_id,
110         membership_type,
111         is_active,
112         purchase_time,
113         activation_time,
114         expiration_time
115     )
116     VALUES (
117         p_purchase_id,
118         p_user_id,
119         p_membership_type,
120         p_is_active,
121         CURRENT_TIMESTAMP,
122         v_activation_time,
```

```
126
127 EXCEPTION
128     WHEN unique_violation THEN
129         RAISE EXCEPTION 'Duplicate value caused a unique violation while purchasing me
130     WHEN OTHERS THEN
131         RAISE EXCEPTION 'Unexpected error in purchase_membership_proc: %', SQLERRM;
132 END;
133 $$;
```

Data Output | Messages | Notifications

```
CREATE PROCEDURE

Query returned successfully in 146 msec.
```

Figure 11. Creating the SP for PURCHASE_MEMBERSHIP_SP

2.4.2 Row-level trigger and function

```
-- Row-level trigger
DROP TRIGGER IF EXISTS trg_check_membership_dates ON bergen.bought_membership;
CREATE TRIGGER trg_check_membership_dates
BEFORE INSERT OR UPDATE ON bergen.bought_membership
FOR EACH ROW
EXECUTE FUNCTION bergen.check_membership_dates();


-- Trigger function that will be used on direct insert-statements for adding to the "b
-- This function will simply raise a notice that an insert statement has been executec
CREATE OR REPLACE FUNCTION bergen.membership_insert_statement_trigger() --(Rikke)
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    RAISE NOTICE 'Insert statement executed on bought_membership table';
    RETURN NULL;
END;
$$;
```

Figure 12. Row-level trigger and function

2.4.3 Statement-level trigger

```
-- Statement-level trigger
DROP TRIGGER IF EXISTS trg_membership_insert_statement ON bergen.bought_membership;
CREATE TRIGGER trg_membership_insert_statement
AFTER INSERT ON bergen.bought_membership
FOR EACH STATEMENT
EXECUTE FUNCTION bergen.membership_insert_statement_trigger();
```

Figure 13. Statement-level trigger

2.4.4 Verifying the insert

```
178  -- Show result
179  -- ------------------------------------------------------------
180  SELECT * FROM bergen.bought_membership;
```

Data Output | Messages | Notifications

Showing rows: 1 to 3 | Page No: 1 of 1

	purchase_id [PK] integer	user_id integer	membership_type character varying (100)	is_active boolean	purchase_time timestamp with time zone (3)	activation_time timestamp with time zone (3)	expiration_time timestamp with time zone (3)
1	19	12	monthly	true	2026-04-13 09:30:04.626+00	2026-04-13 09:30:04.626+00	2026-05-13 09:30:04.626+00
2	20	12	monthly	true	2026-04-13 09:42:55.906+00	2026-04-13 09:42:55.906+00	2026-05-13 09:42:55.906+00
3	21	12	monthly	true	2026-04-13 10:06:50.324+00	2026-04-13 10:06:50.324+00	2026-05-13 10:06:50.324+00

Figure 14. Query verification

2.5 START_TRIP_SP

The **START_TRIP_SP** procedure initiates a new bike trip in the Bcycle Bergen system. It takes the **user_id**, **bike_id**, **program_id**, **start_station_id**, and an optional start time as input parameters, and returns the generated **trip_id** as an **OUT** parameter (**p_trip_id**).

The procedure performs the following steps:

1. Validates that the given station exists under the specified program. If not, an exception is raised.
2. Retrieves the latest bike status record to obtain the current battery level and GPS coordinates.
3. Generates a new unique **ride_id** by incrementing the highest existing trip number.
4. Inserts a new record into **bergen.trip** with all relevant trip data.
5. Inserts a new **bike_status** record marking the bike as in_use.
6. Retrieves the latest **station_status** for the start station.
7. Generates a new **station_status_id** and inserts an updated status row reflecting one fewer available bike and one more available dock.

The **p_trip_id** is defined as an **OUT** parameter so that the caller immediately receives the new trip's ID, which can be used directly in a subsequent call to **END_TRIP_SP**.

2.5.1 Creating the stored procedure

```
DROP PROCEDURE IF EXISTS bergen.start_trip_sp(character varying, character varying, character varying, character varying, timestamp with time zor

CREATE OR REPLACE PROCEDURE bergen.start_trip_sp(
    IN  p_user_id          VARCHAR,
    IN  p_bike_id          VARCHAR,
    IN  p_program_id       VARCHAR,
    IN  p_start_station_id VARCHAR,
    OUT p_trip_id          VARCHAR,
    IN  p_start_time       TIMESTAMPTZ DEFAULT NOW()
)
LANGUAGE plpgsql
AS $BODY$
DECLARE
    v_battery_start  INTEGER;
    v_last_bike_lat  NUMERIC;
    v_last_bike_lon  NUMERIC;
    v_ss_prev        bergen.station_status%ROWTYPE;
    v_new_ssid       VARCHAR;
    v_new_bs_id      VARCHAR;
BEGIN
    -- 1) Check that the station exists under the given program
    PERFORM 1
    FROM bergen.station
    WHERE station_id = p_start_station_id
      AND program_id = p_program_id;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'START_TRIP_SP: station % (program %) does not exist.',
            p_start_station_id, p_program_id;
    END IF;

    -- 2) Retrieve the latest bike status record
    SELECT bs.battery, bs.latitude, bs.longitude
    INTO   v_battery_start, v_last_bike_lat, v_last_bike_lon
    FROM   bergen.bike_status bs
```

```
    -- 2) Retrieve the latest bike status record
    SELECT bs.battery, bs.latitude, bs.longitude
    INTO   v_battery_start, v_last_bike_lat, v_last_bike_lon
    FROM   bergen.bike_status bs
    WHERE  bs.bike_id = p_bike_id
    ORDER  BY bs.last_updated DESC
    LIMIT  1;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'START_TRIP_SP: no status found for bike_id=%.', p_bike_id;
    END IF;

    -- 3) Generate a new ride_id (trip_1, trip_2, ...)
    SELECT 'trip_' || (COALESCE(MAX(CAST(SUBSTRING(ride_id FROM 6) AS INTEGER)), 0) + 1)::TEXT
    INTO   p_trip_id
    FROM   bergen.trip;

    -- 4) Insert the new trip record
    INSERT INTO bergen.trip (
        ride_id, user_id, bike_id, program_id,
        start_time, start_station_id, battery_start
    )
    VALUES (
        p_trip_id, p_user_id, p_bike_id, p_program_id,
        p_start_time, p_start_station_id, v_battery_start
    );

    -- 5) Generate a new bike_status_id (bike_status_1, bike_status_2, ...)
    SELECT 'bike_status_' || (COALESCE(MAX(CAST(SUBSTRING(bike_status_id FROM 13) AS INTEGER)), 0) + 1)::TEXT
    INTO   v_new_bs_id
    FROM   bergen.bike_status;
```

```
    -- 6) Mark the bike as in_use
    INSERT INTO bergen.bike_status (
        bike_status_id, bike_id, status,
        latitude, longitude,
        battery, remaining_range, total_distance, last_updated
    )
    VALUES (
        v_new_bs_id, p_bike_id, 'in_use',
        v_last_bike_lat, v_last_bike_lon,
        v_battery_start, NULL, 0, NOW()
    );

    -- 7) Retrieve the latest station_status for the start station
    SELECT * INTO v_ss_prev
    FROM   bergen.station_status ss
    WHERE  ss.station_id = p_start_station_id
    ORDER  BY ss.last_updated DESC
    LIMIT  1;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'START_TRIP_SP: no station_status found for station_id=%.', p_start_station_id;
    END IF;

    -- 8) Generate a new station_status_id (station_status_1, station_status_2, ...)
    SELECT 'station_status_' || (COALESCE(MAX(CAST(SUBSTRING(station_status_id FROM 16) AS INTEGER)), 0) + 1)::TEXT
    INTO   v_new_ssid
    FROM   bergen.station_status;
```

```
    -- 9) Insert a new station_status row reflecting the departure
    INSERT INTO bergen.station_status (
        station_status_id, station_id, dock_id,
        available_docks,
        regular_bikes_available, electric_bikes_available,
        smart_bikes_available,   cargo_bikes_available,
        is_accepting_returns, is_accepting_renting,
        last_updated
    )
    VALUES (
        v_new_ssid,
        v_ss_prev.station_id,
        v_ss_prev.dock_id,
        v_ss_prev.available_docks + 1,
        GREATEST(v_ss_prev.regular_bikes_available - 1, 0),
        v_ss_prev.electric_bikes_available,
        v_ss_prev.smart_bikes_available,
        v_ss_prev.cargo_bikes_available,
        v_ss_prev.is_accepting_returns,
        v_ss_prev.is_accepting_renting,
        NOW()
    );

    RAISE NOTICE 'START_TRIP_SP: Trip % started for user % with bike % at station %.',
        p_trip_id, p_user_id, p_bike_id, p_start_station_id;
END;
$BODY$;
```

Figure 15. Creating the SP for START_TRIP_SP

2.5.2 Executing the stored procedure

```
-- Test call
CALL bergen.start_trip_sp(
    p_user_id          => 'user_1',
    p_bike_id          => 'bergen_bike_1',
    p_program_id       => 'bcycle_bergen',
    p_start_station_id => 'bergen_station_1',
    p_trip_id          => NULL,
    p_start_time       => NOW()
);
```

Figure 16. Calls the SP

2.5.3 Verifying trip created

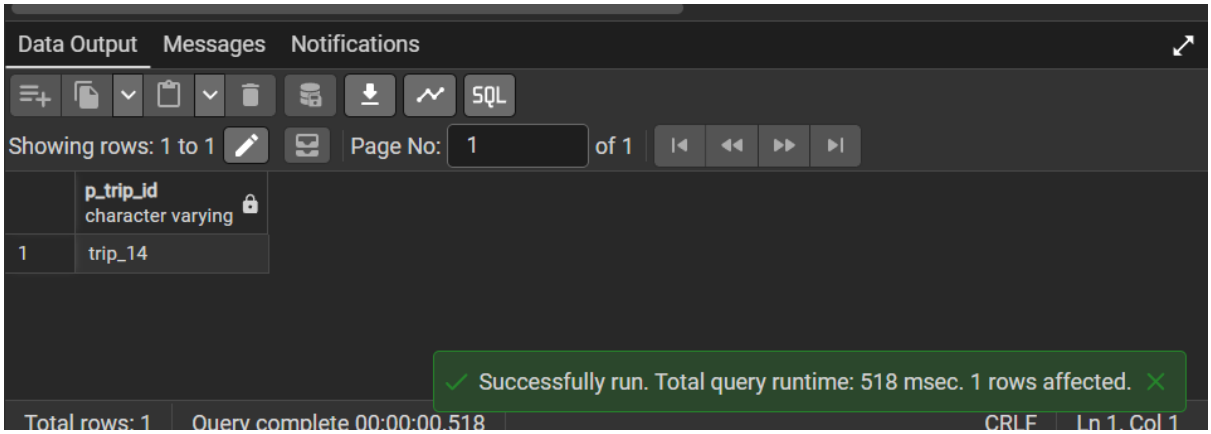

Figure 17. Query verification

Figure 16 shows the result of executing the **START_TRIP_SP** stored procedure. The Data Output panel confirms that the procedure ran successfully, returning **trip_14** as the generated trip identifier through the **p_trip_id** OUT parameter. This demonstrates that the procedure correctly generated a unique trip ID, inserted the corresponding trip record into the **bergen.trip** table, updated the bike status to **in_use**, and updated the station status to reflect the departure.

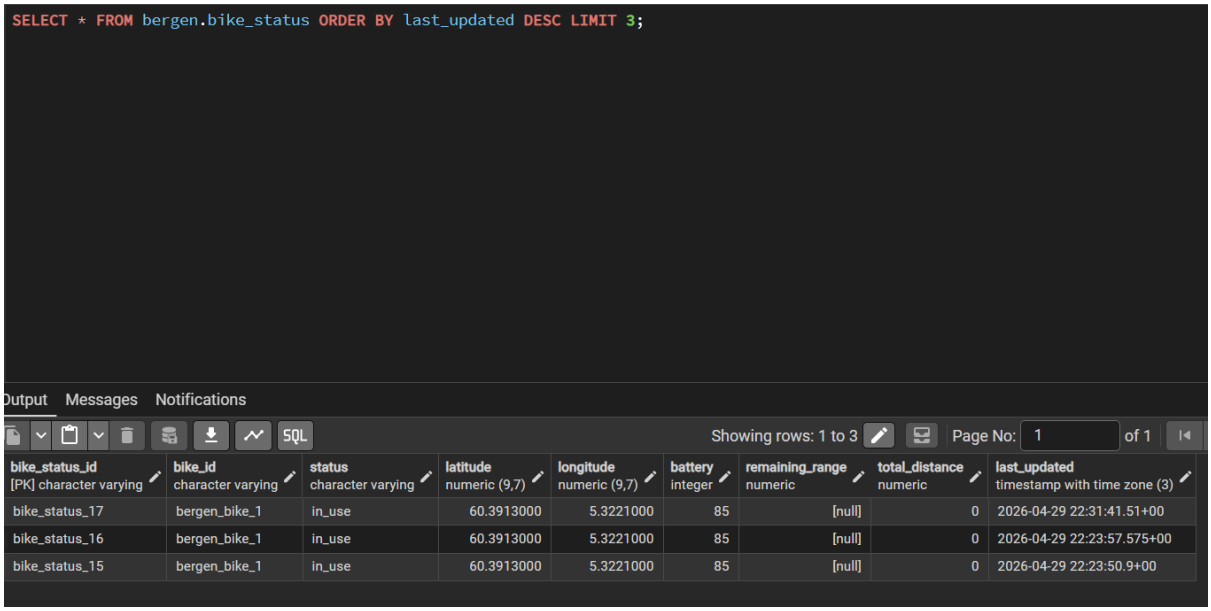

Figure 18. Query verification for bergen.bike_status

Assignment 3: Storage Structure, Data Dictionary, and Performance Analysis

3.1 Physical storage structure

In PostgreSQL, data is stored in tables (relations), which are physically organized into fixed-size blocks called pages. Each page is typically 8 KB in size. Rows are stored inside these pages, and multiple rows can exist within a single page. Indexes and large values are stored separately when necessary (PostgreSQL Global Development Group, n.d).

3.2 Database size

The total size of the database was obtained using a PostgreSQL function. The total database is 8819 KB. This size includes – table data, indexes, internal storage structures. This indicates that the database is relatively small and mainly contains test or development data.

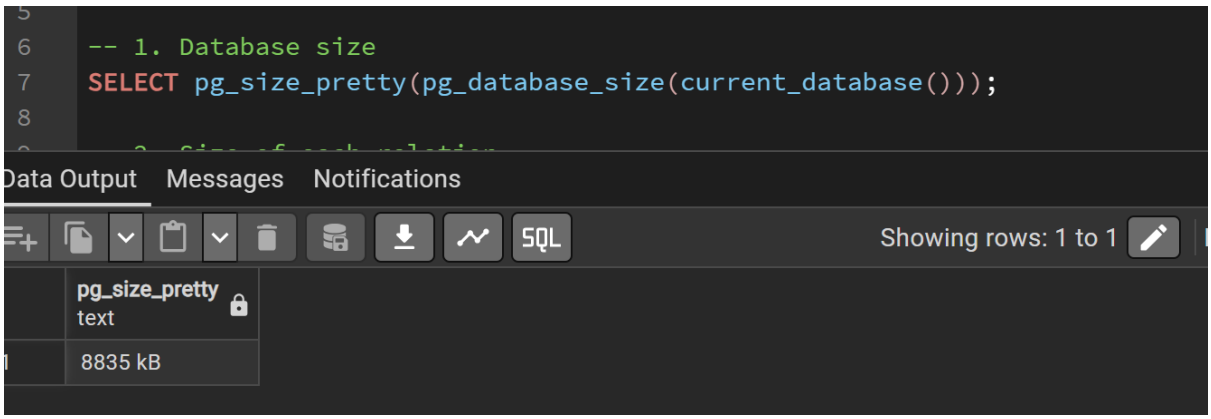

Figure 19. Database size

3.3 Size of each relation

The size of each table in the Bergen schema was measured. Key observations – the largest table is **station_status** (80 KB). Several tables such as **bike_status**, **trip**, **user_auth**, and **user_info** are 48 KB. Smaller tables like program and membership are 24 KB.

```sql
-- 2. Size of each relation
SELECT
    relname AS table_name,
    pg_size_pretty(pg_total_relation_size(c.oid)) AS total_size
FROM pg_class c
JOIN pg_namespace n ON n.oid = c.relnamespace
WHERE relkind = 'r'
  AND n.nspname = 'bergen'
ORDER BY pg_total_relation_size(c.oid) DESC;
```

Data Output | Messages | Notifications

Showing rows: 1 to 12

	table_name name	total_size text
1	station_status	80 kB
2	trip	64 kB
3	bike_status	48 kB
4	user_info	48 kB
5	user_auth	48 kB
6	bike	40 kB
7	station	40 kB
8	station_dock	40 kB
9	bike_type	40 kB
10	bought_members...	40 kB
11	program	24 kB
12	membership	24 kB

Total rows: 12 | Query complete 00:00:00.725

Figure 20. Size of each relation

3.4 Pages and tuples

PostgreSQL stores table data in 8 KB pages (PostgreSQL Global Development Group, n.d). The number of pages and estimated rows for each table were analyzed. Key observations – most tables use 1 page, indicating small data volume. The table station_status uses 2 pages, making it the largest table. Example row counts – bike_status: 26 rows, bike: 22 rows, and user_info 17 rows. The values are estimates generated by PostgreSQL after running the ANALYZE command. Each row is stored within a single page, and multiple tuples are stored per page.

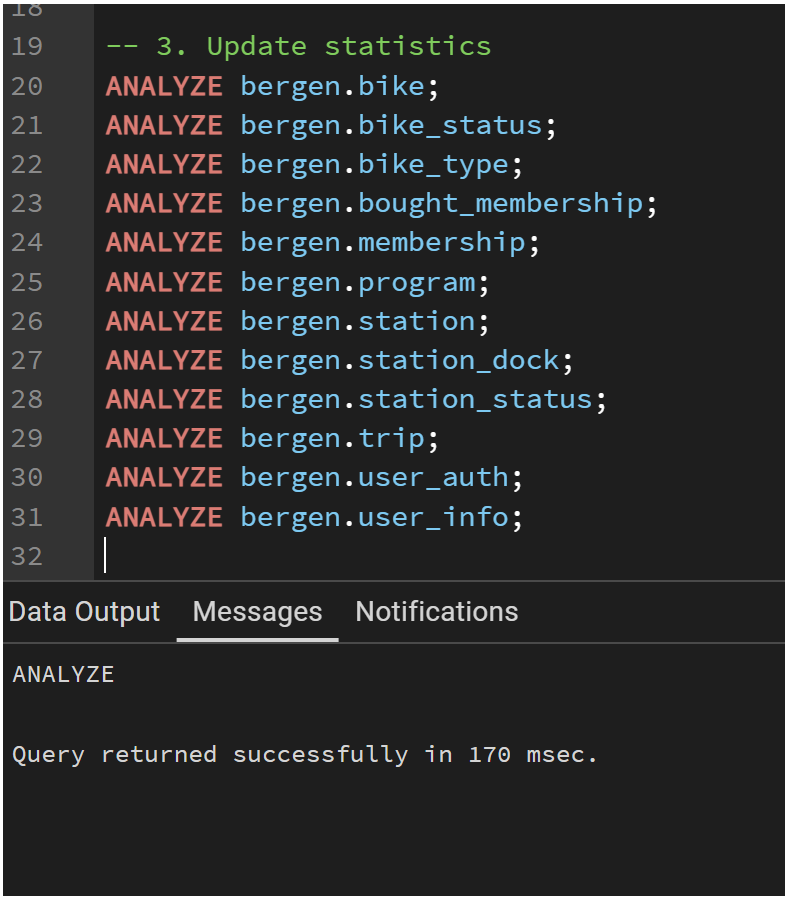

Figure 21. Pages and tuples using ANALYZE command, and updates statistics

```sql
-- 4. Pages and tuples
SELECT
    relname,
    relpages,
    reltuples
FROM pg_class c
JOIN pg_namespace n ON n.oid = c.relnamespace
WHERE relkind = 'r'
  AND n.nspname = 'bergen';
```

Data Output | Messages | Notifications

	relname name	relpages integer	reltuples real
1	bike_type	1	20
2	bike_status	1	26
3	membership	1	6
4	user_info	1	17
5	user_auth	1	16
6	station_status	2	14
7	station	1	6
8	program	1	1
9	bought_members...	1	11
10	trip	1	14
11	station_dock	1	11
12	bike	1	22

Total rows: 12 | Query complete 00:00:00.278

Figure 22. Pages and tuples

3.5 Data dictionary

A data dictionary view was generated using **information_schema.columns**. This shows table names, column names, and data types. Examples for the database are **bike.year_acquired** (INTEGER), **trip.start_time** (TIMESTAMP), **user_info.email** (VARCHAR), and **station.latitude** (NUMERIC). The data dictionary provides a structured overview of the database schema and helps understand how the database is designed.

Query | Query History

```
42
43  -- 5. Data dictionary
44  SELECT
45      table_name,
46      column_name,
47      data_type
48  FROM information_schema.columns
49  WHERE table_schema = 'bergen'
50  ORDER BY table_name, ordinal_position;
```

Data Output | Messages | Notifications

Showing rows: 1

	table_name name	column_name name	data_type character varying
1	bike	bike_id	character varying
2	bike	dock_id	character varying
3	bike	biketype_id	character varying
4	bike	year_acquired	integer
5	bike_status	bike_status_id	character varying
6	bike_status	bike_id	character varying
7	bike_status	status	character varying
8	bike_status	latitude	numeric
9	bike_status	longitude	numeric
10	bike_status	battery	integer
11	bike_status	remaining_range	numeric
12	bike_status	total_distance	numeric
13	bike_status	last_updated	timestamp with time zo…
14	bike_type	biketype_id	character varying

	table_name name	column_name name	data_type character varying
15	bike_type	program_id	character varying
16	bike_type	maker	character varying
17	bike_type	model	character varying
18	bike_type	bike_type	character varying
19	bought_members...	purchase_id	integer
20	bought_members...	user_id	integer
21	bought_members...	membership_type	character varying
22	bought_members...	is_active	boolean
23	bought_members...	purchase_time	timestamp with time zo...
24	bought_members...	activation_time	timestamp with time zo...
25	bought_members...	expiration_time	timestamp with time zo...
26	membership	membership_type	character varying
27	membership	price	numeric
28	membership	duration	integer

	table_name name	column_name name	data_type character varying
29	program	program_id	character varying
30	program	country_code	character
31	program	name	character varying
32	program	location	character varying
33	program	email	character varying
34	program	url	character varying
35	program	time_zone	character varying
36	program	phone	character varying
37	station	station_id	character varying
38	station	program_id	character varying
39	station	name	character varying
40	station	address	character varying
41	station	postal_code	smallint
42	station	latitude	numeric
43	station	longitude	numeric
44	station	capacity	integer
45	station_dock	dock_id	character varying
46	station_dock	station_id	character varying
47	station_dock	bike_id	character varying
48	station_dock	dock_number	integer
49	station_dock	is_occupied	boolean
50	station_dock	is_accepting_returns	boolean

	table_name name	column_name name	data_type character varying
51	station_dock	is_accepting_renting	boolean
52	station_dock	bike_type	character varying
53	station_dock	last_updated	timestamp with time zo…
54	station_status	station_status_id	character varying
55	station_status	station_id	character varying
56	station_status	dock_id	character varying
57	station_status	available_docks	integer
58	station_status	regular_bikes_availab…	integer
59	station_status	electric_bikes_availa…	integer
60	station_status	smart_bikes_available	integer
61	station_status	cargo_bikes_available	integer
62	station_status	is_accepting_returns	boolean
63	station_status	is_accepting_renting	boolean
64	station_status	last_updated	timestamp with time zo…
65	trip	trip_id	integer
66	trip	user_id	integer
67	trip	bike_id	character varying
68	trip	program_id	character varying
69	trip	start_time	timestamp with time zo…
70	trip	end_time	timestamp with time zo…
71	trip	start_station_id	character varying
72	trip	end_station_id	character varying

72	trip	end_station_id	character varying
73	trip	trip_distance	numeric
74	trip	battery_start	integer
75	trip	battery_end	integer
76	trip	trip_cost	numeric
77	trip	trip_duration	interval
78	user_auth	user_id	integer
79	user_auth	hashsalt	character varying
80	user_auth	passhash	character varying
81	user_auth	email	character varying
82	user_info	user_id	integer
83	user_info	name	character varying
84	user_info	surname	character varying
85	user_info	email	character varying
86	user_info	date_of_birth	date
87	user_info	street	character varying
88	user_info	city	character varying
89	user_info	postal_code	smallint
90	user_info	state	character varying
91	user_info	total_trips	integer
92	user_info	creation_date	timestamp with time zo...

Figure 23. Data dictionary

3.6 Query performance analysis

Query | Query History

```
1  EXPLAIN ANALYZE
2  SELECT * FROM bergen.trip
3  WHERE bike_id = 'bergen_bike_1';
4
5  CREATE INDEX idx_trip_bike_id
6  ON bergen.trip(bike_id);
7
8  EXPLAIN ANALYZE
9  SELECT * FROM bergen.trip
10 WHERE bike_id = 'bergen_bike_1';
```

Data Output | Messages | Notifications

	QUERY PLAN text
1	Seq Scan on trip (cost=0.00..1.02 rows=1 width=1392) (actual time=0.009..0.010 rows=1 loop...
2	Filter: ((bike_id)::text = 'bergen_bike_1'::text)
3	Rows Removed by Filter: 1
4	Planning Time: 0.134 ms
5	Execution Time: 0.030 ms

Figure 24. Query performance analysis from bergen.trip

3.6.1 Impact of data types on storage and performance

Data types play an important role in both storage efficiency and query performance. Examples from the Bcycle database:

INTEGER vs BIGINT
columns such as **user_id**, **trip_id**, and **bike_id** use INTEGER, which requires less storage than BIGINT (PostgreSQL Global Development Group, n.d.). This reduces row size and improves performance.

VARCHAR vs CHAR
Most text fields (**name**, **email**, **model**) use VARCHAR, which stores only the actual length of the value (PostgreSQL Global Development Group, n.d.). This is more space-efficient than fixed-length CHAR.

- TIMESTAMP vs TEXT
Columns such as **start_time** and **end_time** use TIMESTAMP, allowing efficient filtering, sorting, and time-based calculations (PostgreSQL Global Development Group, n.d.). Using appropriate data types reduces storage usage and improves query execution speed.

3.6.2 Impact of indexes and performance

Indexes improve query performance by reducing the number of pages PostgreSQL needs to scan. Examples from the Bcycle database:

Primary keys such as – **bike_id**, **trip_id**, and **station_id** are automatically indexed.

Queries such as – **SELECT * FROM trip WHERE bike_id = 'B1';** are faster because the database can use an index instead of scanning the entire table. The benefits of using indexes are due to its faster search operations, faster joins between tables, and improved filtering performance. The costs of using indexes are additional storage space, and slower insert and update operations. Indexes are essential for performance but must be used carefully to balance storage and efficiency (PostgreSQL Global Development Group, n.d.)

3.7 User administration

3.7.1 Read-Only Role

A role that can only read the table's data in the schema.

```
-- Role: bcycle_user
-- Pass: bcycle_user_pass
-- DROP ROLE IF EXISTS bcycle_user;
CREATE ROLE bcycle_user WITH LOGIN NOSUPERUSER INHERIT NOCREATEDB
NOCREATEROLE NOREPLICATION NOBYPASSRLS
PASSWORD 'bcycle_user_pass';
GRANT pg_read_all_data TO bcycle_user WITH INHERIT OPTION, SET OPTION;
-- GRANT SELECT ON ALL TABLES IN SCHEMA bergen TO bcycle_user; =>
Alternative way to give read access to
```

Figure 25. Read-only role for Bcycle user

3.7.2 Procedure Execution Role

A role that can execute all procedures in the schema.

```
-- Role: bcycle_admin
-- Pass: bcycle_admin_pass
-- DROP ROLE IF EXISTS bcycle_admin;
CREATE ROLE bcycle_admin LOGIN NOSUPERUSER INHERIT NOCREATEDB
NOCREATEROLE NOREPLICATION NOBYPASSRLS
PASSWORD 'bcycle_admin_pass';
GRANT bcycle_user TO bcycle_admin WITH INHERIT OPTION, SET OPTION;
-- ALTER DEFAULT PRIVILEGES IN SCHEMA bergen GRANT EXECUTE ON ROUTINES
TO bcycle_admin; => Be able to automatically get access to future
procedures.
GRANT EXECUTE ON ALL FUNCTIONS IN SCHEMA bergen TO bcycle_admin;
```

Figure 26. Procedure execution role for Bcycle admin

3.7.3 Procedure Execution Role for Account, Membership and Trip

A role that can execute only the procedures to create a new account, a new membership and create a new trip.

```
-- Role: account_admin
-- Pass: account_admin_pass
-- DROP ROLE IF EXISTS account_admin;
CREATE ROLE account_admin LOGIN NOSUPERUSER INHERIT NOCREATEDB
NOCREATEROLE NOREPLICATION NOBYPASSRLS
PASSWORD 'account_admin_pass';
GRANT bcycle_user TO account_admin WITH INHERIT OPTION, SET OPTION;
GRANT EXECUTE ON PROCEDURE bergen.create_account_sp TO account_admin;
GRANT EXECUTE ON PROCEDURE bergen.purchase_membership_proc TO
```

Figure 27. Procedure Execution Role for account, membership, and trip

3.7.4 Difficulties Encountered During Role Creation

There was not a specific problem with the creation of the account administrator's role. However, we did find issues running the procedures different roles due to the roles not being allowed to alter tables. This made it so that we needed to alter the procedures to allow the roles to run them as a user with rights to edit tables.

```
-- Alter PROCEDURE so the different roles can use it
ALTER PROCEDURE bergen.purchase_membership_proc
security definer SET search_path = postgres, pg_temp;

ALTER PROCEDURE bergen.create_account_sp
security definer SET search_path = postgres, pg_temp;

ALTER PROCEDURE bergen.start_trip_sp
security definer SET search_path = postgres, pg_temp;

ALTER PROCEDURE bergen.create_station_sp
security definer SET search_path = postgres, pg_temp;

ALTER PROCEDURE bergen.create_bicycle_proc
security definer SET search_path = postgres, pg_temp;
```

Figure 28. Alter procedure for different procedures

3.7.5 Additional Individual and Group Roles

We have created four roles that we consider important: **customer_support**, **station_manager**, **operations_tech_team**, and **system_admin**. In terms of individual and group roles, we decided that **system_admin** is assigned as an individual role, while **operations_tech_team**, **customer_support** and

station_manager are considered group roles.

- **customer_support**: The staff needs support 24/7 and needs read-only access to user information, trip history, and membership logs to assist users with inquiries. However, they should not be able to modify any data directly, as changes must go through procedures.

- **system_admin**: System admin requires full access to all tables and sequences in the **bergen** schema. This role will particularly manage the database, perform backups, and handle the system-level operations. This role is specifically assigned to skilled personnel.

- **station_manager**: Responsible for managing physical station operation, including registering new docks and updating station availability. They need to write access to station tables, but should not be able to modify user accounts or trip logs.

- **operations_tech_team**: The operations tech team is responsible for managing the physical infrastructure of the bike share system, including registering new bikes, updating station docks and monitoring availability. Need to write access to operational tables, but they should not have access to user accounts or financial-related data.

3.7.6 Privilege Grants

The GRANT command has two variants: one that grants privileges on a database object, and one that grants membership in a role (PostgreSQL, 2026). To enforce least-privilege access in the Bergen schema, we assigned role-specific permissions based on operational responsibilities. The following roles were evaluated: **station_manager**, **customer_support**, **operations_tech_team**, and **system_admin**. Privilege verification was performed using **information_schema.role_table_grants** (table privileges) and **information_schema.usage_privileges** (sequence privileges). This ensured that both data access and object usage rights were checked in a structured and auditable way.

- **customer_support**: Granted mainly read privileges (**SELECT**) on support-relevant tables (e.g., user, membership, and trip-related data). No direct write privileges **(INSERT/UPDATE/DELETE**) were granted on operational infrastructure tables. Sequence access is limited to what is strictly required.

- **station_manager:** Granted operational privileges on station-related tables (**typically SELECT**, and where required **INSERT/UPDATE**) for station availability and dock management. Restricted from modifying user account and financial/membership records unless explicitly needed.

- **operations_tech_team:** Granted technical operational privileges on infrastructure objects (bike/station/dock/status tables), including required write privileges (**INSERT/UPDATE)** for maintenance workflows. Access to user/account-sensitive data is restricted.

- **system_admin**: Granted broad administrative privileges across schema objects, including table-level management privileges and sequence privileges (**USAGE**). This role is intended for trusted administrative users only.

Sequence privileges (**USAGE**) were granted only to roles that need to execute inserts/procedures that depend on sequence-backed identifiers. The reason why he chose this type of role model is that it separates support, operations, and administration duties, reduces accidental or unauthorized data changes, and improves security by ensuring each role receives only the privileges required for its job function.

3.7.7 Specify one privilege that you find important to grant to all users using the Bcycle database system

One important privilege that should be granted to all users of the Bcycle database system is the ability to read data from the tables. This ensures that all roles, including the **bcycle_user** account, can access and verify the information stored in the database. Additionally, other roles such as administrators and developers need read

access to validate their work, troubleshoot issues, and ensure data consistency. Providing read access improves transparency and allows users to effectively interact with the system without compromising data integrity. Read access is the most basic privilege that any user who interacts with the system should have; if they do not have this, they probably do not need direct access to the database.

4. Justifications

For the design of the database, we tried to follow a logic to make the design simple and concise. Most attributes are NOT NULL and use constraints to make sure there is consistent data across the data. Our biggest choice we made was to make the tables "**bike_status**" and "**station_status**" into log tables that create a new row every time there is a change to stations and the stations docks or bikes. This creates a log to follow changes with each row giving a good insight into the state of the different units, this was advised by the TA's in the class.

Moreover, the individual role **system_admin** is designated due to full administrative access to the database, and it should be restricted to a single trusted and skilled person or a very small group of authorized technical employees. Granting broad administrative privileges to a group role would increase the risk of human failure or unauthorized changes to critical database assets. In terms of the privileges, the **system_admin** is granted broad administrative privileges across all schema objects, including full table-level management privileges and sequence privileges (USAGE).

This is necessary because the system admin is responsible for the overall integrity and maintenance of the system, including managing stored procedures, trigger functions, sequences, and schema-level objects. Reasoning for sequence privileges is that the admin may need to reset, inspect, or modify sequence counters directly, for instance, to correct auto-increment values after failed insert attempts. Hence, this access level is justified only for a trusted administrative user and should never be assigned to general staff.

Moving on to the group roles, especially **customer_support**, this role is a group role since it requires multiple staff members within the customer support department and shares the same access requirements. All customer support representatives need to view the same categories of data in order to assist users with inquiries or tickets, whereas a shared role is both practical and efficient. Additionally, customer_support is granted mainly read privileges (SELECT) on support-relevant tables (user table, membership records, and trip data).

This is justified because customer support staff need to look up user accounts, verify membership status, and review trip history in order to resolve user complaints and inquiries. No direct write privileges (INSERT, UPDATE, DELETE) are granted on operational infrastructure tables, as customer support staff should not be able to modify bike statuses, station availability, or dock records, since these fall outside of their scope of responsibility. Sequence access is limited to what is strictly required, since customer support staff do not perform inserts that depend on sequence-backed IDs. This restricted access model ensures that sensitive operational and financial data cannot be accidentally or intentionally misconfigured by support staff.

station_manager is also set as a group role because all station managers across the Bergen bike program share the same operational responsibilities and therefore require the same level of database access. Defining a shared group role ensures consistency in access control across all managers without needing to configure privileges individually. For grant verification, granted operational privileges on station-related tables (**SELECT** and **INSERT/UPDATE**) for station availability and dock management.

This is justified because station managers are responsible for registering new stations and docks, updating dock availability, and ensuring that **station_status** accurately reflects real-world conditions. In other words, they are intentionally restricted from modifying user account data and financial or membership records, as these fall under the responsibility of the user management and finance teams.

Lastly, the **operations_tech_team** role is designated as a group role because all members of the technical operations team share the same infrastructure

responsibilities. The reasoning is that whether registering new bikes, updating bike statuses after maintenance, or monitoring dock availability, all members require the same set of technical privileges to perform their work efficiently.

With the necessary write privileges (INSERT, UPDATE), this is justified because the team is responsible for the physical management of the bike fleet and station infrastructure, meaning it requires the ability to register new bikes, update their status, and reflect maintenance outcomes in the system. User access and account-sensitive data are intentionally restricted, as the tech team has no legitimate need to view or modify personal user information or financial records. Sequence privileges (USAGE) are granted to this role where necessary, as team members perform inserts that depend on sequence-backed IDs such as **bike_id_seq** when registering new bikes into the system.

5. Discussion

For the **CREATE_ACCOUNT_SP**, the consideration of implementing password hashing would be a critical factor in order to improve and strengthen our security, but it is outside of this scope. Currently, we have passhash and passsalt values as strings and stored directly under the **user_auth** table without any cryptographic encoding. If we had this database in production, then passhash and passsalt values would be stored as ciphertext input using BCrypt. In that way, this would be used to hash passwords combined with a randomly generated salt before storage.

Since both triggers in the database use **AFTER**, it is important to set a distinction between **BEFORE** and **AFTER** to justify the choice.

For instance, a **BEFORE** trigger fires before the operation is applied to the system (PostgreSQL, 2026). In other words, this means the row has not yet been implemented to the table when the function executes. A prime example is the **gen_station_id()** trigger, which uses **BEFORE INSERT** to generate the **station_id** before the row is written. When **AFTER** being utilized, the row would already be implemented without a valid ID.

In our system, both triggers use **AFTER** (**trg_set_initial_bike_status**, **trg_log_station_dock_summary**). The reasoning behind this, especially **trg_set_initial_bike_status**, was that the **bike_id** must already exist in the bike table before it can be referenced as a foreign key in **bike_status**. Hence, using **BEFORE** would probably risk insertion into **bike_status** before the bike record is fully implemented, and it will potentially cause a foreign key violation.

For **trg_log_station_dock_summary**, **AFTER** is also required because the summary log must be able to recalculate and reflect all changes made by that statement. Using **BEFORE** would mean the summary is calculated before the new data is written, which will result in producing inaccurate availability counts.

For the **trip** table, a potential column could be for the polyline that shows the trips route, so the user can see afterwards and for Bcycle to track where the bike has been. The polyline can also be useful to calculate the distance of the trip.

GRANT EXECUTE ON ALL PROCEDURES IN SCHEMA bergen TO bcycle_admin; will only grant rights to execute existing procedures to **bcycle_admin**. If we wanted to give the role the ability to execute current and future procedures, we could use: **ALTER DEFAULT PRIVILEGES IN SCHEMA bergen**
GRANT EXECUTE ON FUNCTIONS TO bcycle_admin; (PostgreSQL: Documentation: 18: ALTER DEFAULT PRIVILEGES) However we chose to not as it was not needed or asked of us in the assignment.

Start-Procedure

One important design consideration that emerged during the implementation of the stored procedures relates to how inserts into **bike_status** and **station_status** are currently handled. In the current implementation, these inserts are written directly inside the stored procedures themselves. While this approach works correctly within the scope of the procedures we have implemented, it introduces a limitation when considering reusability and maintainability across the broader system.

If we were to implement additional stored procedures, such as a **STOP_TRIP_SP** to handle the ending of a trip, the same insert and update logic for **bike_status** and **station_status** would need to be duplicated inside that procedure as well. This violates the principle of keeping logic in a single place and increases the risk of inconsistencies if the logic ever needs to be updated. A change made in one procedure would need to be manually replicated in every other procedure that performs the same operation, which is error-prone and difficult to maintain at scale.

A better approach would be to convert these insert and update operations into dedicated trigger functions. By attaching triggers to the **bike** and **station_dock** tables, any direct insert or update on those tables would automatically fire the corresponding trigger, which would then handle the status updates consistently and automatically. This means that regardless of whether the operation originates from a stored procedure, a direct SQL statement, or a future procedure we have not yet written, the status tables would always be updated correctly without requiring any additional logic in the calling code.

This approach would significantly improve reusability, since the trigger function acts as a single source of truth for how status is maintained. It would also improve robustness, because the status update logic cannot be accidentally skipped or forgotten when writing new procedures. The system would behave consistently whether a bike's dock assignment changes through **START_TRIP_SP**, **STOP_TRIP_SP**, or any direct operational update performed by a station manager or maintenance role.

In hindsight, implementing these operations as trigger functions from the beginning would have been a more scalable design decision. It is a refinement we would prioritize in the future iteration of this system.

Conclusion

This project highlights the design and implementation of a relational database system for the Bcycle bike-sharing platform. The solution demonstrates a structured approach to database development, where conceptual modeling, logical design, and physical considerations are aligned to support real-world operational requirements for completing this final assignment.

An important factor of the system is the distinction between static entities and time-dependent data. The use of dedicated status tables with timestamps enables both real-time functionality and historical analysis. This provides valuable insight into system behavior over time and supports operational decision-making.

Moreover, the implementation ensures strong data integrity through constraints, transactions, and database-level logic. The use of pgSQL centralizes business rules and ensures consistent behavior across all operations. Additionally, appropriate data types and indexing strategies support efficient storage and query performance.

However, some limitations remain. Certain logic is duplicated across stored procedures, which reduces maintainability. The group managed to minimize the duplicates to improve work efficiency, query performance, and data integrity. Some of the work was outside the scope of the assignment but was kept in mind for future real-world solutions.

To conclude this project, the database provides a solid and scalable foundation. The design balances clarity, performance, and consistency, and it is suitable for further development and extension. This project could not be completed without an outstanding team effort and shared knowledge, where we learned together through trial and error.

Group contribution

Ask:

Created the stored procedure and trigger functions to automate the creation of new stations. Created the full SQL-script. Created the entity relationship diagram for the schema. Created some of the user roles and granted them privileges. Went through all stored procedures and functions to test that they worked properly and change them if there were any issues.

Signature:

Jimmy:

Established the stored procedure to automate the creation of new user accounts. Had also been responsible for creating one individual role and some group roles that are considered important, in addition to granting them privileges. Had responsibility for structuring the final portfolio, finalizing the document and assignment submission.

Signature:

Rikke:

Create the stored procedure to automate the creation of new memberships. created one row-level statement and one statement-level trigger. Me and Ida also did the

storage analysis, which included storage structure, data dictionary, and performance analysis.

Signature:

Ida:
My contribution to the assignment consisted of designing and implementing a stored procedure and corresponding triggers for managing bicycles. Furthermore, me and Rikke collaborated on the analysis of the database's physical storage analysis, including evaluation of the storage structure, data dictionary and performance.

Signature:

Madalitso:
Was responsible for the **START_TRIP_SP** stored procedure. This included writing the full PL/pgSQL procedure, debugging multiple syntax and schema compatibility issues (undeclared variables, incorrect column names, missing primary key generation, and parameter ordering constraints in PostgreSQL), and verifying the result against live data in pgAdmin 4. The procedure was successfully tested and confirmed to insert a new trip record, update bike status to **in_use**, and update station status accordingly.

Signature:

Madalitso Skjelnes

Sources

PostgreSQL Global Development Group. (n.d.).*TOAST*.
https://www.postgresql.org/docs/current/storage-toast.html
Retrieved 21.04.26.

PostgreSQL Global Development Group. (n.d.). *Indexes*.
https://www.postgresql.org/docs/current/indexes-intro.html
Retrieved 27.04.26.

PostgreSQL Global Development Group. (n.d.). Alter Default Privileges.
https://www.postgresql.org/docs/current/sql-alterdefaultprivileges.html
Retrieved 27.04.26.

PostgreSQL Global Development Group. (n.d.). *Numeric types*.
https://www.postgresql.org/docs/current/datatype-numeric.html
Retrieved 29.04.26

PostgreSQL Global Development Group (n.d.). *Character types*.
https://www.postgresql.org/docs/current/datatype-character.html
Retrieved 29.04.26

PostgreSQL Global Development Group (n.d.). *Date/time type*.
https://www.postgresql.org/docs/current/datatype-datetime.html
Retrieved 29.04.26

PostgreSQL Global Development Group (n.d.). *Indexes*.
https://www.postgresql.org/docs/current/indexes.html
Retrieved 29.04.26

PostgreSQL Global Development Group (n.d.). *GRANT*.
https://www.postgresql.org/docs/current/sql-grant.html
Retrieved 30.04.26

PostgreSQL Global Development Group (n.d.). *37.1. Overview of Trigger Behavior*. https://www.postgresql.org/docs/current/trigger-definition.html
Retrieved 30.04.26